

ChronoContext

A Temporal Cognition Layer for Large Language Models

Author: Alan Friar

Organization: EnBra Group

Version: 1.1

Date: 2026-03-15

Abstract

Large Language Models (LLMs) process sequences of tokens but do not maintain an explicit representation of real-world time progression. While models can interpret temporal expressions embedded within text, they lack persistent awareness of elapsed time between events or interactions.

This paper introduces ChronoContext, a lightweight temporal cognition layer designed to augment LLM systems with chronological awareness. ChronoContext detects events in conversational input, stores them as timeline anchors, computes temporal relationships, and injects temporal context into prompts before they are processed by a language model.

The architecture functions as middleware and requires no modification of the underlying LLM. By providing structured temporal primitives such as event anchors, timeline graphs, and temporal indexing, ChronoContext enables improved reasoning about processes that evolve over time and across conversational sessions.

1. Introduction

Large Language Models have demonstrated significant advances in natural language processing and reasoning. However, most current LLM systems operate using static textual context rather than continuous temporal state.

In everyday interactions, users frequently reference events occurring over time. Statements such as:

- "I started a new diet three days ago."
- "It was my wife's birthday last week."
- "The project deadline is next month."

require reasoning about durations, sequences, and timelines.

While LLMs can interpret temporal language within a single prompt, they do not maintain persistent awareness of when events occur relative to one another across conversations. As a result, temporal reasoning must be reconstructed from text rather than derived from an explicit timeline.

ChronoContext addresses this limitation by introducing a temporal cognition layer that tracks events, maintains timelines, and injects chronological context into LLM prompts.

2. Temporal Blindness in LLM Systems

Transformer-based language models represent input as sequences of tokens. Positional encoding provides information about the order of tokens in a sequence but does not represent real-world time.

Consequently, LLM systems lack several capabilities required for chronological reasoning:

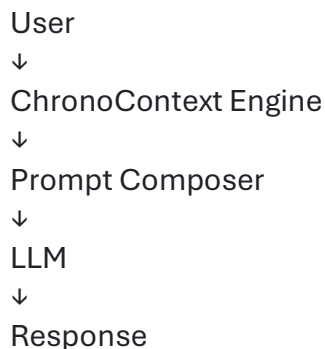
- persistent tracking of events
- computation of elapsed time between events
- recognition of temporal relationships between actions

This limitation can be described as temporal blindness.

ChronoContext introduces explicit temporal primitives to address this gap.

3. ChronoContext Architecture

ChronoContext operates as middleware between the user and the language model.



The ChronoContext engine processes temporal information before each prompt is sent to the model.

4. Temporal Primitives

ChronoContext introduces several core temporal variables.

Core variables:

- `current_time`
- `conversation_start`
- `last_message_time`

Derived variables:

- `conversation_elapsed`
- `time_since_last_message`
- `time_since_event`
- `time_between_events`

These primitives provide the foundation for chronological reasoning.

5. Event Anchors

Events are represented as anchors within a timeline.

Each event anchor contains:

- `event_id`
- `entity_id`
- `event_type`
- `timestamp`
- `description`
- `category`
- `confidence`
- `source`

Example:

diet_change - 2026-03-03

protein shake breakfast routine started

Event anchors provide structured reference points for reasoning about changes over time.

6. Event Detection

ChronoContext includes an event detection component that identifies temporal events within conversational text.

Detection signals include:

- temporal phrases such as “three days ago” or “last week”
- action verbs indicating state changes such as “started”, “stopped”, or “scheduled”

Detected events are converted into timeline anchors and stored in the system timeline.

7. Timeline Graph Model

Events are stored as nodes in a directed temporal graph.

Example sequence:

```
diet_started  
↓  
weight_gain  
↓  
doctor_visit
```

Edges represent relationships between events and include time intervals between them.

This structure enables reasoning about sequences, durations, and event progression.

8. Temporal Memory Across Sessions

ChronoContext maintains persistent timeline memory that extends beyond a single conversational session.

Event anchors may be stored and referenced across multiple interactions, allowing systems to maintain awareness of processes that evolve over longer periods of time.

For example, an event such as:

diet_started - March 3

may be referenced in later conversations days or weeks later without requiring the user to repeat the original event description.

This cross-session temporal continuity allows LLM-assisted systems to reason about ongoing activities, habits, projects, and timelines in a way that more closely resembles human chronological reasoning.

9. Temporal Reasoning

The temporal primitives defined earlier enable higher-level chronological reasoning within the ChronoContext system.

The temporal reasoning engine calculates relationships between events.

Examples include:

- time_since_event
- time_between_events
- event_sequence
- milestone_progress

Example computation:

Diet start → weight gain = 3 days

These relationships are incorporated into prompt context.

10. Temporal Context Injection

Before sending a prompt to the LLM, ChronoContext generates a temporal context block.

Example:

Temporal Context

Current date: 2026-03-06

Timeline

- Protein shake routine started: 3 days ago
- Weight increase reported: today

Temporal relationship

- Diet change → weight gain = 3 days

This structured context allows the language model to reason chronologically without requiring changes to the underlying model architecture.

11. Applications

A prototype ChronoContext middleware demonstration was executed locally on 15 March 2026 to validate the architectural concepts described in this paper.

The prototype detected event anchors within conversational input, stored them as persistent timeline entries, and generated structured temporal context injected into the prompt structure prior to LLM execution.

The demonstration confirmed the feasibility of the ChronoContext temporal cognition architecture, including:

- event anchor detection
- persistent timeline memory
- cross-session temporal continuity
- temporal context injection

A record of the prototype execution is included in this repository.

12. Applications

ChronoContext enables improved reasoning in domains where time progression is important.

Examples include:

- health monitoring
 - habit tracking
 - project management
 - compliance monitoring
 - personal assistance
-

13. Conclusion

ChronoContext introduces a temporal cognition layer for LLM systems that enables chronological reasoning without modifying the underlying model architecture. By detecting events, maintaining timelines, and injecting temporal context into prompts, ChronoContext allows language models to reason more effectively about processes that evolve over time and across multiple conversational sessions.